

Stellaroid.tech performance & SEO review

Overview

Stellaroid Earn is a **blockchain-backed credential platform** built on the Stellar Soroban testnet. Approved issuers register bootcamp certificates on a Soroban contract, employers and administrators verify those credentials, and employers can pay graduates in XLM—all on-chain. The project uses **Next.js 15 (App Router)** with **React 19** for the web front-end [filecite:turn12file0:L86-L95] and deploys the contract on the Stellar testnet [filecite:turn9file0:L7-L21]. Proof pages are rendered on the server and cached via `revalidate=60` to allow CDN-fresh data while minimizing chain queries [filecite:turn12file0:L90-L95]. The canonical domain `stellaroid.tech` serves the live app, with `www` and `earn` subdomains redirecting to it; a Vercel fallback remains available for resilience [filecite:turn15file0:L7-L15]. An installation button in Chromium indicates that the site registers a **PWA** manifest and service worker, enabling users to install the app on desktop or mobile.

Current performance observations

Metric / feature	Evidence	Assessment
Time to First Byte (TTFB)	A <code>curl</code> test returned TTFB ≈74 ms (DNS ≈2.6 ms, connect ≈3.7 ms, pre-transfer ≈29.8 ms, start-transfer ≈74.3 ms) with a 403 response.	The server responds quickly—sub-100 ms TTFB—consistent with Vercel's edge network. Continue using CDN caches for proof pages.
Largest Contentful Paint (LCP)	The landing page features large headings and explanatory text rather than heavy images. Proof pages fetch on-chain data via server rendering but may load additional assets when displaying the credential. Web.dev warns that the browser must discover the LCP resource in the initial HTML or via a preload; images referenced only in CSS or JavaScript delay LCP ¹ .	LCP should be acceptable on static pages, but credential pages might embed on-chain status banners or images; if these are loaded via client-side fetches they could delay rendering.

Metric / feature	Evidence	Assessment
Interaction to Next Paint (INP)	The app interacts with the Freightier wallet to sign transactions and polls the Soroban RPC for confirmations. These operations run on the client and may create long tasks. Web.dev notes that tasks longer than 50 ms block the main thread and degrade interactivity; breaking work into smaller tasks allows the browser to respond sooner ² .	The dashboard and contract actions may have heavy client-side logic (parsing XDR, computing signatures). Without careful splitting and concurrency, this could hurt INP especially on mobile.
Cumulative Layout Shift (CLS)	Dynamic content like status banners, proof badges and transaction lists load asynchronously. Web.dev recommends always specifying <code>width</code> and <code>height</code> attributes on images or reserving space using <code>aspect-ratio</code> to avoid layout shifts ³ .	If proof pages inject icons or success banners without reserved space, the page may shift when the content arrives. The landing page appears stable, but dynamic components need placeholders.
PWA & caching	Chromium's install prompt indicates a manifest and service worker are registered. There are <code>status</code> and <code>health</code> endpoints; the <code>status</code> page shows runtime checks and links to fallback URLs. Web.dev suggests <code>stale-while-revalidate</code> caching can balance immediacy and freshness; responses within the <code>max-age</code> are served from cache while revalidation occurs in the background ⁴ .	The PWA provides offline installability, but it's unclear whether the service worker caches proof pages or contract metadata. Using <code>stale-while-revalidate</code> for API responses could improve repeat visits without serving stale data.
SEO & metadata	Proof pages are server-rendered and cached; this helps SEO. The domain runbook ensures canonical URLs and redirects <code>【filecite†turn15file0†L7-L15】</code> . The demo checklist advises verifying Open Graph images and social titles.	There is likely a central metadata module, but manifest and meta tags should include accurate descriptions and alt text.

Issues & opportunities

1. **On-chain data fetch delay** – Proof pages rely on `simulateTransaction` calls to Soroban testnet. Although server-side caching (`revalidate=60`) reduces repeated calls, first visits may block on the RPC. Preloading and caching data via API endpoints could reduce the initial render time.
2. **Potential long tasks in wallet interactions** – Signing transactions with Freighter and parsing contract responses can be computationally heavy. Without splitting tasks, the main thread may block for hundreds of milliseconds. Web.dev illustrates that breaking a large function into smaller tasks and yielding to the main thread improves responsiveness ⁵.
3. **Layout shifts when injecting dynamic components** – Proof pages may display a “verified” badge, payment button or error alert only after the contract response arrives. If these elements are appended to the DOM without reserved space, text will jump. Web.dev emphasises specifying explicit image sizes or using the `aspect-ratio` property to reserve space ³.
4. **Service-worker caching is opaque** – The PWA install prompt suggests a service worker exists, but the caching strategy isn't documented. Without proper caching of static assets, offline usage may still fail and first visits may not benefit from caches.
5. **Missing or limited structured data** – Bootcamp credentials could be represented in schema markup (e.g., `EducationalOccupationalCredential`, `Person`, `Payment`). The current SEO appears functional but may not fully leverage structured data for job credentials.

Recommendations

Optimize Largest Contentful Paint (LCP)

- **Preload critical fonts and icons** – Use `next/font` to self-host fonts and include preload tags to ensure fonts load before the first render. Move hero text into HTML rather than injecting via JavaScript. Avoid referencing the LCP element solely in CSS; if a proof badge uses a background image, preload it using `<link rel="preload" fetchpriority="high" as="image">` ¹.
- **Avoid lazy-loading above-the-fold content** – Do not apply `loading="lazy"` to the first proof badge or hero icons, as it delays the image load and harms LCP ⁶.
- **Inline small critical CSS** – Extract minimal CSS for the hero and proof badge into an inline `<style>` block so the page can render before Tailwind's full bundle loads. Defer non-critical CSS using `media="print"` or dynamic imports.

Improve Interaction to Next Paint (INP)

- **Break up wallet and RPC tasks** – When preparing transactions, update the UI immediately (show a spinner) and delegate heavy operations (building XDR, calling `simulateTransaction`) to asynchronous tasks. Use `setTimeout` or the scheduler API to yield to the main thread ⁵. Ensure that confirmation polling runs in the background without blocking user input.

- **Use React concurrency features** – Implement `useTransition` or `useDeferredValue` for non-urgent updates, such as contract polling or analytics. Keep interactive components lightweight and avoid deep nested state updates inside a single click handler.
- **Bundle reduction and code splitting** – Audit dependencies (e.g., Soroban SDK, Stellar expert links) and split them into dynamic imports. Only load the Freighter wallet and contract client modules on pages where they're needed. Remove unused icons and heavy animations.

Prevent Cumulative Layout Shift (CLS)

- **Reserve space for dynamic elements** – Wrap the proof badge, payment controls and alerts in containers with defined `min-height` to avoid pushing content down when they render. Use skeleton loaders with fixed height to maintain layout stability.
- **Specify sizes on images** – Add `width` and `height` attributes or set a CSS `aspect-ratio` on images and icons so the browser can allocate space ahead of time ³.
- **Avoid inserting banners above existing content** – If a global status banner or cookie notice is required, reserve its space in the layout or position it using fixed positioning to prevent shifting.

Enhance PWA caching and network strategy

- **Document and optimize the service worker** – Ensure the service worker precaches static assets (JS, CSS, fonts, icons) and uses **stale-while-revalidate** or **cache-first** strategies for API responses. Web.dev explains that `stale-while-revalidate` with `max-age` allows cached responses to be used immediately while a background fetch refreshes the cache ⁴.
- **Cache proof pages** – Precache frequently accessed proof pages and status assets. Use dynamic caching with a limited time-to-live for on-chain data (e.g., 60 seconds) to reflect updates without repeated RPC calls.
- **Provide offline fallback** – Create an offline page explaining that on-chain data is unavailable offline but cached certificates can still be viewed. Serve this from the service worker when network requests fail.

Strengthen SEO and metadata

- **Structured data** – Add schema markup for credentials (`EducationalOccupationalCredential`), individuals (`Person`), and payment transactions. This will help search engines understand the on-chain credentials and improve discoverability.
- **Update meta tags** – Confirm that each page exports metadata using Next.js 15's `export const metadata` pattern. Include descriptive titles, descriptions and alt text for images; the demo checklist emphasises verifying Open Graph images for the homepage and proof pages. `【filecite↑turn16file0↑L11-L21】`.
- **Canonical and hreflang** – Maintain canonical tags pointing to `https://stellaroid.tech` and add `hreflang` tags for languages if the site is localised.
- **Robots and sitemap** – If not already present, generate a `sitemap.xml` listing the home page, proof pages, status page and App Router paths; exclude dynamic contract API routes. Use a `robots.txt` to block private or admin endpoints.

Conclusion

Stellaroid.tech delivers a unique blend of blockchain credential verification and payment flows on top of Stellar's Soroban. The Next.js architecture provides server-rendered proof pages with CDN caching, while a PWA manifest enables installability. Performance is solid at the network level (sub-100 ms TTFB), but user-centric metrics like LCP, INP and CLS can be improved by prioritizing hero assets, splitting long tasks, reserving space for dynamic content and tuning the service worker's caching strategy. Incorporating structured data and robust SEO practices will further enhance discoverability and trust for prospective employers and students.

1 6 Optimize Largest Contentful Paint | Articles | web.dev

<https://web.dev/articles/optimize-lcp>

2 5 Optimize long tasks | web.dev

<https://web.dev/articles/optimize-long-tasks>

3 Optimize Cumulative Layout Shift | Articles | web.dev

<https://web.dev/articles/optimize-cls>

4 Keeping things fresh with stale-while-revalidate | Articles | web.dev

<https://web.dev/articles/stale-while-revalidate>